

Open in app ↗



Medium



Search



Write



Comprehensive Guide to Handling Apache Spark Errors & Performance Issues: An AI Engineer's Playbook

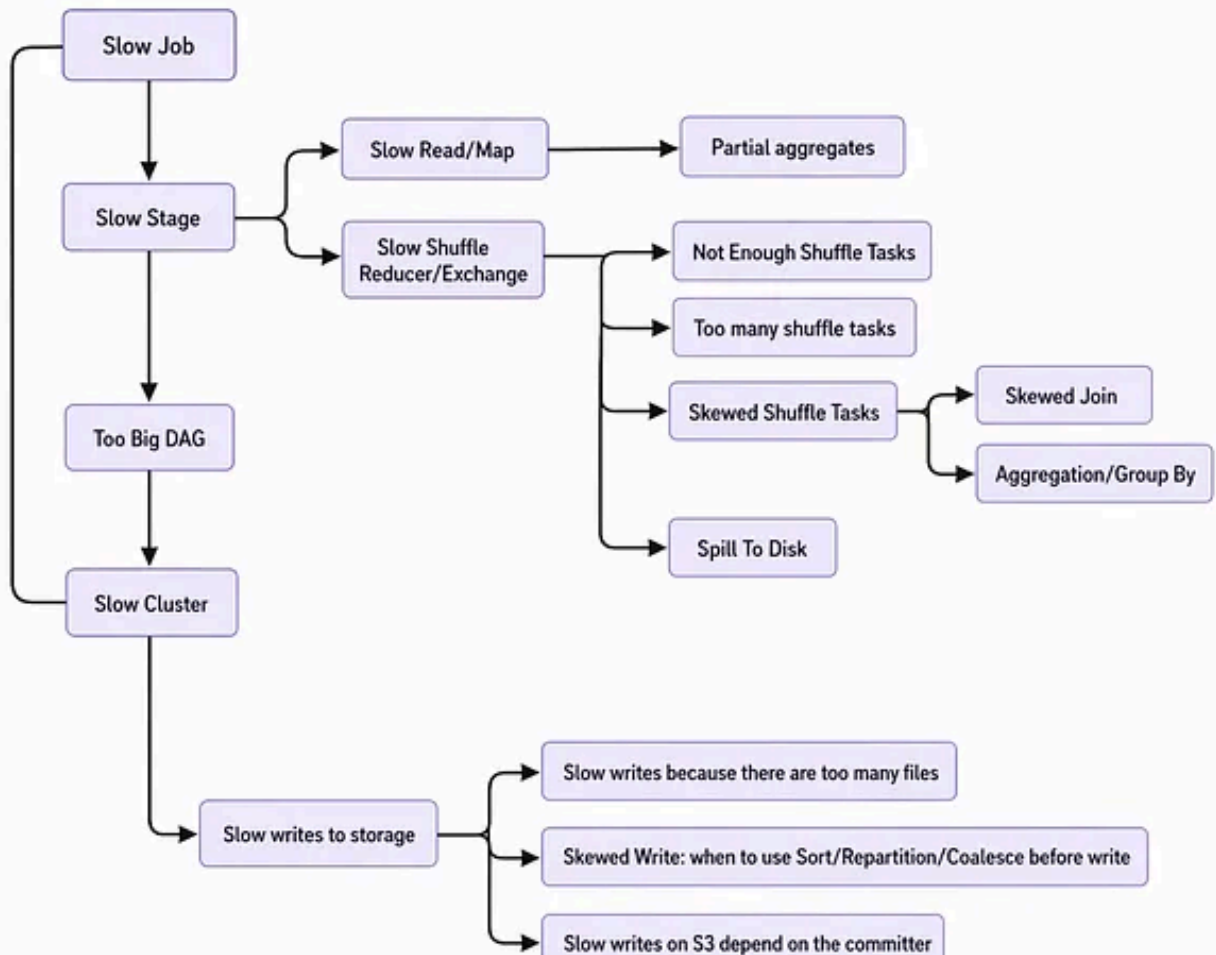
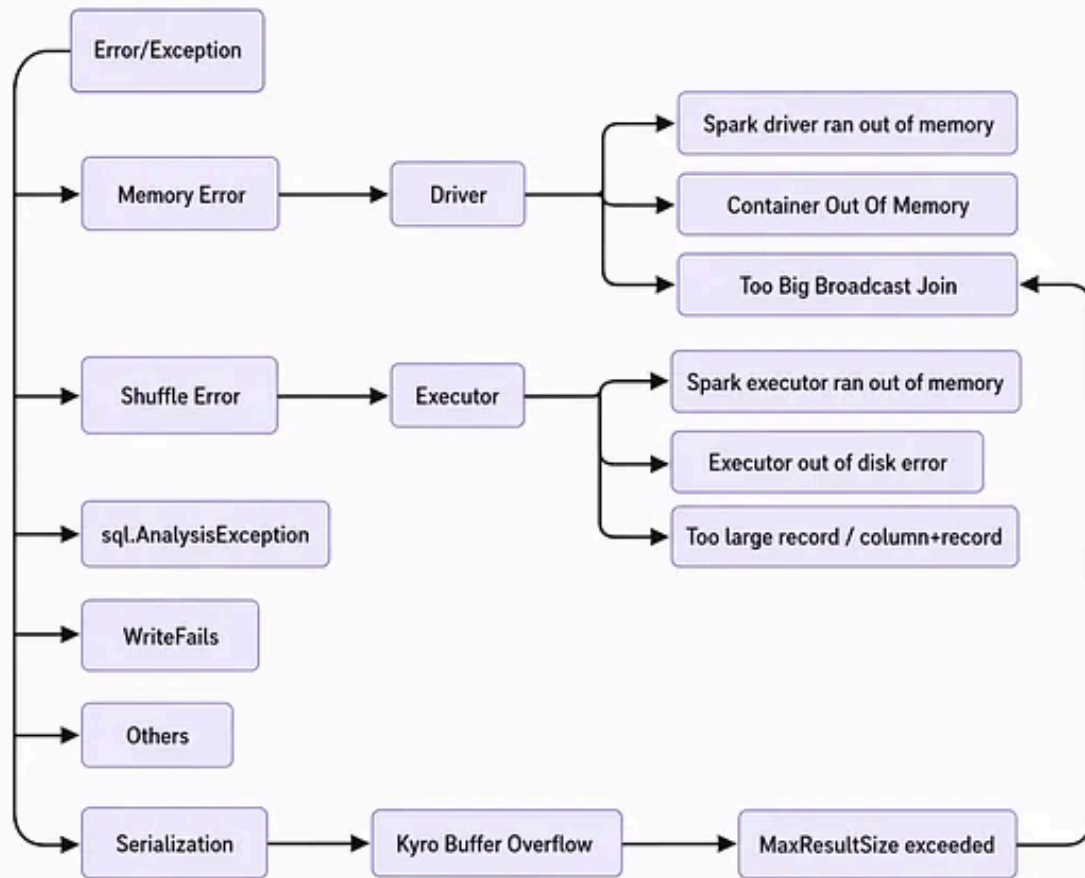


Anurag Sharma · 6 min read · Apr 24, 2026



2





As AI engineers building large-scale data pipelines (whether for feature engineering, recommendation systems, or batch inference), Apache Spark is our daily driver. But Spark jobs fail. Often dramatically. The two mind maps you shared capture the most common failure modes and performance bottlenecks in production Spark workloads.

This blog follows a **structured AI-engineer workflow** for every error/issue shown in the diagrams:

1. **What is the error/issue?** (Symptoms + real-world impact)
2. **Potential causes** (root triggers, with Spark config/code examples)
3. **How an AI Engineer handles it**
 - **Identify** (logs, UI, tools)
 - **Cure** (immediate fix)
 - **Add a debug mode/prevention layer** (so it never happens again in your CI/CD or production pipelines)

We'll cover **every node** from both diagrams in logical groups.

1. Memory Errors

1.1 Spark Driver Ran Out of Memory

What is it? The driver JVM throws `java.lang.OutOfMemoryError: Java heap space` (or GC overhead limit exceeded). The entire application dies because the driver is the coordinator.

Potential causes

- `df.collect()`, `df.toPandas()`, or `df.show()` on huge DataFrames.
- Large broadcast variables or accumulators.
- Driver holding too much metadata (thousands of tasks/partitions).
- Default `spark.driver.memory=1g` (way too low for modern workloads).
- Complex Catalyst optimizer plans in Spark SQL.

AI Engineer Workflow Identify

- Driver logs (first place) + Spark UI → “Environment” tab → Driver memory.
- Spark History Server → Memory graph spikes on driver.
- `jstack` or `jmap` on the driver pod if you have shell access.

Cure:

- Increase: — `driver-memory 8g` (or `spark.driver.memory=8g`).
- Refactor: replace `collect()` with `write` + limit sampling, or use
`spark.sql.adaptive.enabled=true`
`spark.sql.adaptive.coalescePartitions.enabled=true`.

Prevent (Debug Mode): Add this to every job:

```
import os
if os.getenv("SPARK_DEBUG_MODE") == "true":
    df = df.sample(0.01) # force tiny sample in debug
```

```
print(f"DEBUG: DataFrame size before collect = {df.count()}")  
spark.conf.set("spark.driver.memory", "8g") # or read from env
```

Set up Prometheus + Grafana alert:

“Driver heap usage > 80% → kill & restart with 2x memory”.

1.2 Spark Executor Ran Out of Memory

What is it? Executors throw OOM. Tasks fail with “ExecutorLostFailure” or “Task failed due to OutOfMemoryError”.

Potential causes

- Huge partitions (skew or bad repartition).
- spark.executor.memory too low (default 1g).
- Off-heap memory not configured (spark.memory.offHeap.enabled).
- Large UDFs or Python UDFs (PySpark) holding data in the Python heap.

AI Engineer Workflow Identify

- Spark UI → Executors tab → Memory graph + “Failed” tasks.
- Look for “ExecutorLost” in the driver log with reason “Memory”.

Cure:

- spark.executor.memory=6g + spark.executor.memoryOverhead=2g.
- Enable Adaptive Query Execution (AQE): spark.sql.adaptive.enabled=true.

- Use `spark.sql.files.maxPartitionBytes` to control partition size.

Prevent (Debug Mode):

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
if os.getenv("SPARK_DEBUG_MODE") == "true":
    spark.conf.set("spark.sql.adaptive.coalescePartitions.minPartitionNum", "400")
```

Add custom metric: `log spark.task.metrics.memoryUsed` per stage.

1.3 Container Out Of Memory (YARN/K8s)

What is it? YARN/Kubernetes kills the container (Container exited with non-zero code or Killed by YARN for exceeding memory limits).

Potential causes

- `spark.executor.memoryOverhead` too low (default 10% of executor memory).
- Python processes (PySpark) using more memory than Java heap reports.
- Off-heap usage (Arrow, GPU, etc.).

AI Engineer Workflow Identify

- YARN ResourceManager UI or `kubectl describe pod`.
- Look for “Killed by YARN” or OOMKilled in pod events.

Cure: Increase overhead: `spark.executor.memoryOverhead=3g` (or 20% rule of thumb).

Prevent:

```
spark.conf.set("spark.executor.memoryOverhead", "4096m")
spark.conf.set("spark.kubernetes.memoryOverheadFactor", "0.2")
```

In debug mode, run with `— conf spark.executor.cores=2` to reduce per-container footprint.

1.4 Executor Out of Disk Error

What is it? `ExecutorOutOfDisk` or shuffle spill fails because the local disk is full.

Potential causes

- Heavy shuffle spill (`spark.shuffle.spill.diskWriteBufferSize`).
- Too many small files written to local disk (`spark.local.dir`).
- No cleanup of previous runs.

AI Engineer Workflow Identify

- Executor logs: “No space left on device”.
- Spark UI → Storage tab (spill metrics).

Cure: Increase `spark.local.dir` to multiple high-capacity disks or use faster NVMe.

Prevent:

```
spark.conf.set("spark.local.dir", "/mnt/ssd1,/mnt/ssd2")  
spark.conf.set("spark.shuffle.spill.compress", "true")
```

Debug mode: force early spill with lower `spark.memory.fraction`.

2. Broadcast Join Issues

2.1 Too Big Broadcast Join

What is it? `BroadcastExchange` fails or exceeds limit → “Cannot broadcast the table that is larger than...” or OOM on driver.



Potential causes

- `spark.sql.autoBroadcastJoinThreshold` (default 10MB) is too low for your table.
- Table grows unexpectedly (no stats).

AI Engineer Workflow Identify

- Spark UI → SQL tab → “BroadcastExchange” stage shows huge size.

Cure:

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "100MB") # or -1 to disable  
df1.join(broadcast(df2)) # explicit
```

Prevent: Debug mode auto-detect:

```
if os.getenv("SPARK_DEBUG_MODE") == "true":  
    if df2.count() > 1_000_000:  
        print("DEBUG: Switching to Sort-Merge Join")  
        spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")
```

3. Record Size Issues

3.1 Too Large Record / Column+Record

What is it? SparkException: Cannot write ... because it exceeds ... or shuffle fails on a single massive row.

Potential causes

- JSON/XML columns with huge nested data.
- Base64 images or embeddings in one column.

AI Engineer Workflow Identify

- Stack trace contains “record is too large”.

Cure: `spark.sql.files.maxRecordsPerFile` + split the column or use `spark.sql.parquet.compression.codec`.

Prevent: Add schema validation step + debug mode that runs `df.select(max(length(col("huge_col"))))`.

4. Serialization Errors

4.1 Kryo Buffer Overflow

What is it? `KryoBufferOverflowException` — object too large for serializer buffer.

Potential causes

- Complex Python objects or custom classes in RDDs.
- Default `spark.kryo.serializer.buffer=64k` too small.

AI Engineer Workflow Identify

- Exact exception in executor logs.

Cure:

```
spark.conf.set("spark.kryo.serializer.buffer", "256m")
```

```
spark.conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
```

Prevent: Register custom classes in Kryo and add debug-mode size checks on UDF return values.

4.2 MaxResultSize Exceeded

What is it? MaxResultSizeExceededException — driver result too big (usually from collect).

Potential causes

- spark.driver.maxResultSize (default 1g) hit.

AI Engineer Workflow Identify

- Clear message in driver log.

Cure: Increase to 8g or refactor to avoid a large collection.

Prevent: Same debug-mode sample + guard clause as driver OOM.

5. Shuffle Errors

What is it? Shuffle fetch failed, fetch failed, or OOM during shuffle.

Potential causes:

- Data skew, too few/large partitions, insufficient shuffle service memory.

AI Engineer Workflow Identify:

- Spark UI → “Shuffle Read/Write” metrics + red tasks.

Cure: Enable AQE + `spark.sql.adaptive.skewJoin.enabled=true`.

Prevent: Debug mode forces higher `spark.sql.shuffle.partitions=400`.

6. SQL AnalysisException

What is it? `org.apache.spark.sql.AnalysisException: ... cannot resolve ... or column ambiguity`.

Potential causes

- Typos, missing aliases after joins, and wrong schema.

AI Engineer Workflow Identify

- Exact SQL line in the exception.

Cure Print schema + use `df.alias("t1")`.

Prevent Debug mode from running `spark.sql("EXPLAIN EXTENDED " + query)` and asserting no unresolved columns.

7. WriteFails

What is it? Write stage fails (permission, format, partition issues, S3 rate limits, etc.).

Potential causes

- Wrong S3 credentials, too many small files, committer issues.

AI Engineer Workflow Identify

- The last stage in UI is “SaveIntoDataSourceCommand”.

Cure: Use `spark.sql.parquet.compression.codec=snappy` + `coalesce/repartition` before write.

Prevent: Always use `spark.conf.set(“spark.sql.sources.commitProtocolClass”, “org.apache.spark.sql.execution.datasources.SQLHadoopMapReduceCommitProtocol”) for S3.`

8. Performance Issues / Slow Jobs (Second Mind Map)

8.1 Slow Read/Map

Causes: Small files, bad compression, S3 list operations. **Cure:** `spark.sql.files.maxPartitionBytes=512m`, `coalesce` input. **Prevent:** Debug mode prints the `inputFiles` count.

8.2 Slow Shuffle / Reducer / Exchange

Causes: Not enough shuffle tasks, too many, skew. **Cure:** AQE + `spark.sql.shuffle.partitions` tuned dynamically. **Prevent:** Debug mode logs skew stats (`df.groupBy(spark_partition_id()).count()`).

8.3 Skewed Shuffle Tasks / Skewed Join / Aggregation/Group By

Classic data skew. **Cure:** Salting + AQE skew join. **Prevent:** Add `skewHint` or pre-repartition on high-cardinality key.

8.4 Spill To Disk

Cure: Increase memory fraction or use AQE. **Prevent:** Debug mode lowers the memory fraction to force early spill detection.

8.5 Slow Writes to Storage

Sub-issues: Too many files, skewed write, S3 committer. **Cure:** repartition(200) or coalesce before write +
`spark.hadoop.mapreduce.fileoutputcommitter.algorithm.version=2`.
Prevent: Debug mode always runs `df.write.option("maxRecordsPerFile", 100_000)`.

8.6 Too Big DAG / Slow Cluster / Slow Stage

Cure: Break job into smaller stages, increase cluster size, or use `spark.scheduler.mode=FAIR`. **Prevent:** Debug mode runs `df.explain(mode="extended")` and fails the job if there are > 500 nodes in the DAG.

Final AI Engineer Checklist (Copy-Paste into Every Spark Job)

```
debug = os.getenv("SPARK_DEBUG_MODE") == "true"

spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.driver.maxResultSize", "4g")

if debug:
    print("=== SPARK DEBUG MODE ENABLED ===")
    spark.sparkContext.setLogLevel("DEBUG")
    # force sampling, extra explain(), size checks
```

Monitor with **Spark UI + History Server + Prometheus + Grafana + custom job metadata** (data volume, partition count, skew metrics) stored in a central “job health” table.

With this systematic **Identify → Cure → Prevent** loop, your Spark jobs will go from “randomly failing every Friday night” to “reliably production-grade AI data infrastructure.”

Happy debugging!

[Ai Data Engineer](#)[Data Engineering](#)[Big Data](#)[Spark](#)[Data Analytics](#)

Written by Anurag Sharma

[Edit profile](#)

98 followers · 4 following

Data Engineering Specialist with 10+ exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!